

Homework 3

Matthew Hutchinson

November 4, 2025

Task: Design a path planner that produces control inputs $x_c(t), y_c(t), \theta_c(t)$ so that the middle node tracks the prescribed $(x^*(t), y^*(t))$ under gravity.

Method Description

This section provides a clear step-by-step description of the numerical method used to simulate and control the beam, including pseudocode for force and energy evaluation, time integration, boundary condition enforcement, and the PI control law.

(i) Force and Energy Evaluation

The dynamics are computed using a Newton–Raphson iteration to solve for the nodal positions $\mathbf{q}$ at each time step. The total residual force includes contributions from elastic deformation, inertia, and gravity. The updated positions $\mathbf{q}_{\text{new}}$ minimize the residual:

- **Elastic force and stiffness:** Computed using axial and bending energy models.
- **Inertial force**
- **External forces:** Gravity is applied to the y -components of each node.

Pseudocode:

Start with initial guess $\mathbf{q}_{\text{new}} = \mathbf{q}_{\text{old}}$

Repeat until convergence:

1. Compute inertial force and Jacobian:
 $\mathbf{F}_{\text{inertia}} = m/dt * ((\mathbf{q}_{\text{new}} - \mathbf{q}_{\text{old}})/dt - \mathbf{u}_{\text{old}})$
 $\mathbf{J}_{\text{inertia}} = m\mathbf{Mat} / dt^2$
2. Compute elastic force and Jacobian:
 $(\mathbf{F}_{\text{elastic}}, \mathbf{J}_{\text{elastic}}) = \text{compute_elastic_forces}(\mathbf{q}_{\text{new}})$
3. Form residual and Jacobian:
 $\mathbf{R} = \mathbf{F}_{\text{elastic}} - \mathbf{F}_{\text{inertia}} - \text{gravity}$
 $\mathbf{J} = \mathbf{J}_{\text{elastic}} + \mathbf{J}_{\text{inertia}}$
4. Solve for update:
 $d\mathbf{q} = \text{solve}(\mathbf{J}_{\text{free}}, -\mathbf{R}_{\text{free}})$
 $\mathbf{q}_{\text{new}}[\text{free}] += d\mathbf{q}$
5. Check convergence:
if $||d\mathbf{q}|| < \text{tolerance} \rightarrow \text{done}$

(ii) Time Stepping

Time integration is performed using the backward Euler method. After computing $\mathbf{q}_{\text{new}}$, the velocity is updated and logged.

Pseudocode:

For each time step:

1. Solve for q_{new} using Newton-Raphson (objfun)
2. Apply Dirichlet constraints (control inputs)
3. Compute velocity: $u_{\text{new}} = (q_{\text{new}} - q_{\text{old}}) / dt$
4. Update: $q_{\text{old}} = q_{\text{new}}$, $u_{\text{old}} = u_{\text{new}}$

(iii) Enforcement of Dirichlet Constraints

Dirichlet boundary conditions are applied to the last two nodes of the beam to prescribe both position and orientation. Node N is fixed at (x_c, y_c) , and node $N - 1$ is offset by ΔL in the direction defined by the control angle θ_c .

Pseudocode:

Given control inputs (x_c, y_c, θ_c):

Set node N (tip):

```
q[2*N]      = x_c
q[2*N + 1]  = y_c
```

Set node $N-1$:

```
q[2*(N-1)]  = x_c - ΔL * cos(\theta_c)
q[2*(N-1) + 1] = y_c - ΔL * sin(\theta_c)
```

(iv) Control Law: Mapping Error to (x_c, y_c, θ_c)

A proportional-integral (PI) controller is used to steer the middle node of the beam along a desired trajectory. The controller minimizes the error between the actual and target position of the middle node, as well as the error in tip orientation.

Let the tracking error be:

$$\mathbf{e}(t) = \begin{bmatrix} x_{\text{mid}} - x_{\text{mid}}^* \\ y_{\text{mid}} - y_{\text{mid}}^* \\ \theta_c - \theta^*(t) \end{bmatrix}$$

Pseudocode:

Initialize $\text{error_sum} = [0, 0, 0]$

Initialize $\theta_c = 0$

For each time step:

1. Compute errors:
 $\text{error} = [\text{mid_actual} - \text{mid_desired}, \theta_c - \theta_{\text{star}}]$
2. Integrate and smooth derivative:
 $\text{error_sum} += \text{error} * dt$
 $\text{error_deriv} = \alpha * \text{prev_deriv} + (1 - \alpha) * (\text{error} - \text{prev_error}) / dt$
3. Compute control input:
 $\text{delta_u} = K_p * \text{error} + K_i * \text{error_sum} + K_d * \text{error_deriv}$
 $\text{delta_u} = \text{clip}(\text{delta_u}, \text{min_bounds}, \text{max_bounds})$
4. Apply control:
 $x_c += -\text{delta_u}[0]$
 $y_c += -\text{delta_u}[1]$
 $\theta_c += -\text{delta_u}[2]$
5. Store logs and update state

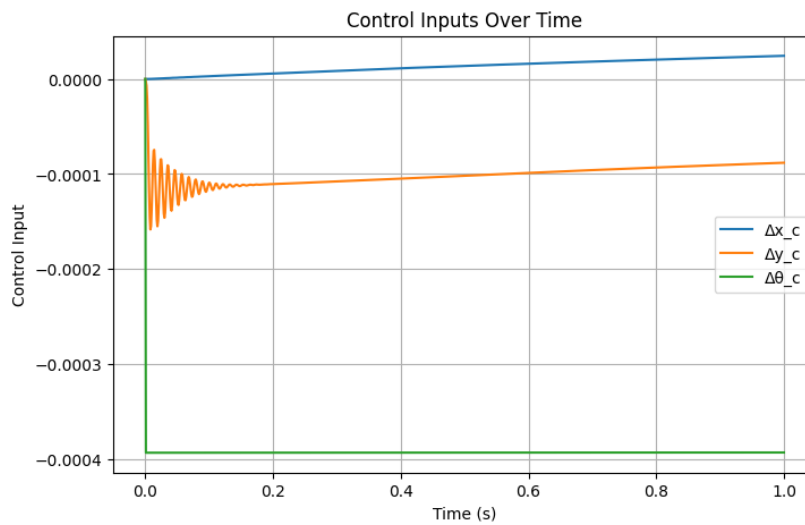
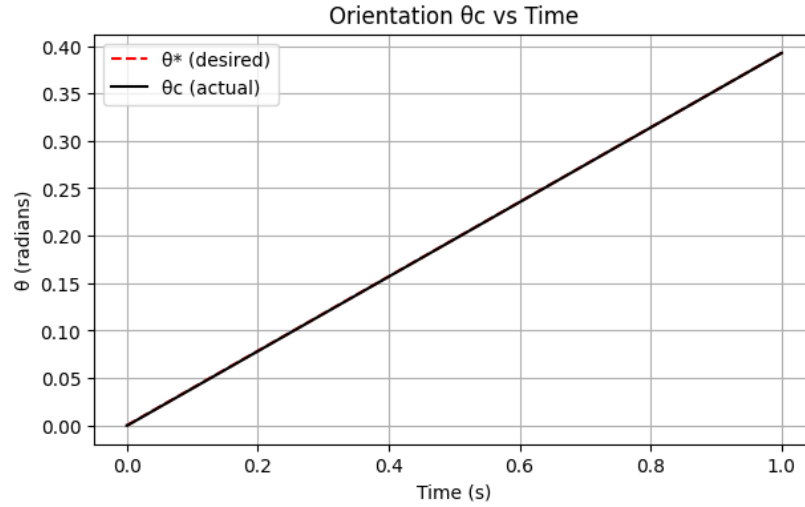
Plots of the control inputs x_c, y_c, θ_c over time.

My PID gains are:

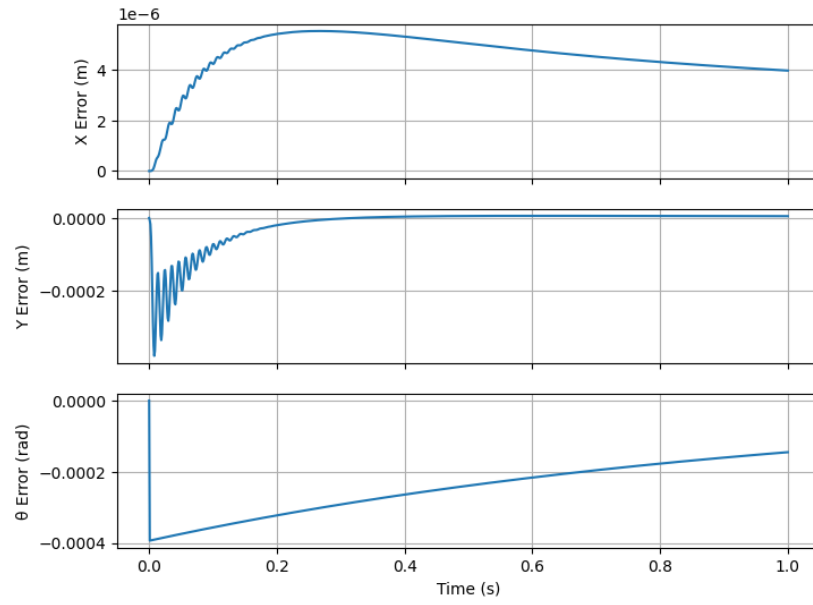
```
Kp = np.array([0.4, 0.4, 1.0])  
Ki = np.array([5.0, 5.0, 1.0])  
Kd = np.array([0.0, 0.0, 0]) # D term to noisy
```

θ_c sweeps clockwise from 0 to $\pi/8$

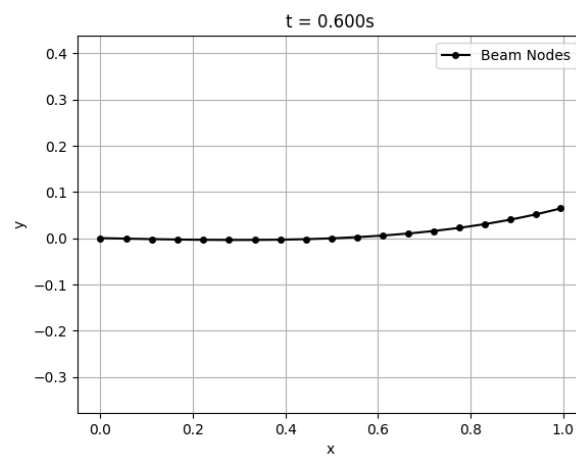
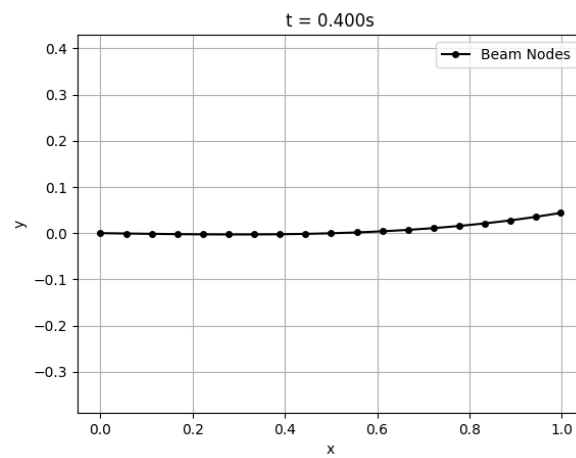
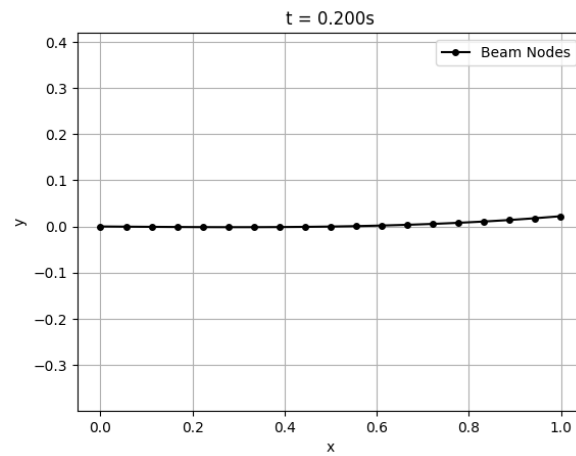
```
theta_star = np.linspace(0, np.pi / 8, Nsteps)
```

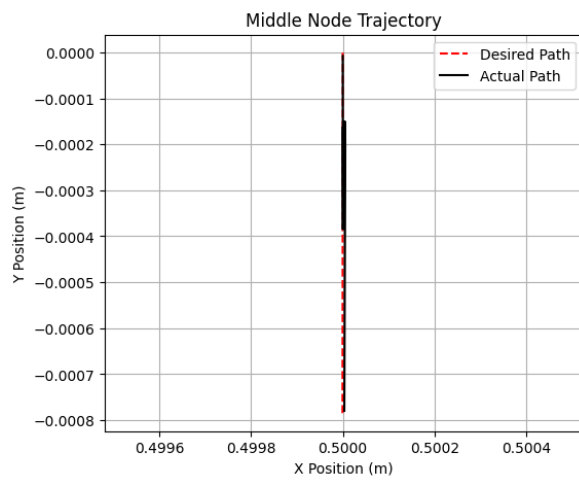
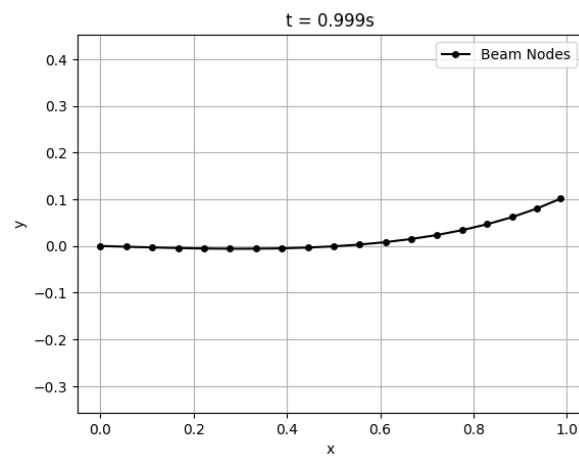
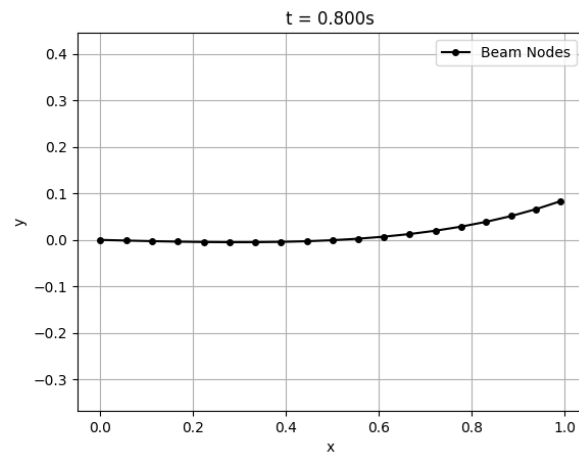


Tracking Error Over Time

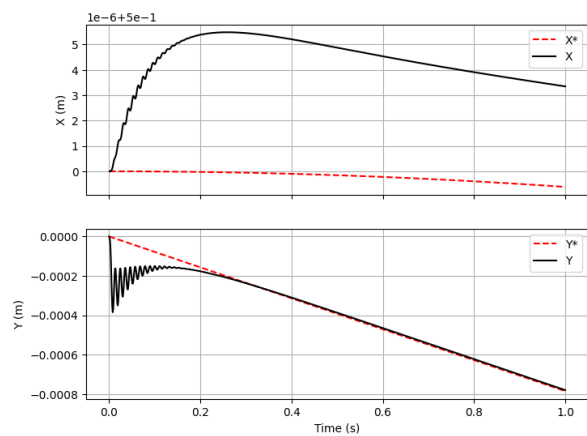


Five snapshots of the beam shape (node positions) during the motion.





Middle Node Position vs. Time



A brief discussion of feasibility limits due to the robot’s workspace and joint limits; explain how you handle infeasible commands (e.g., saturation or trajectory re-timing).

The beam system exhibits physical constraints arising from its limited workspace and boundary-actuated control. Specifically, the control inputs (x_c, y_c, θ_c) correspond to the tip position and orientation of the beam, which are indirectly mapped to the internal nodes through elastic deformation. This introduces two primary feasibility limits:

- **Workspace Constraints:** The reachable workspace of the beam is bounded by its finite length L and the geometry of the structure. Since the beam cannot stretch or fold arbitrarily, the middle node cannot be commanded to follow paths that would require the tip to move beyond the beam’s arc-length limit.
- **Orientation Limits:** The desired tip angle θ_c must remain within a reasonable range to avoid overbending or inducing large internal forces that could lead to numerical instability.

Handling Infeasible Commands:

To maintain physical realism and numerical stability, the following mechanisms are used to handle infeasible control commands:

- **Command Saturation:** Control inputs $(\Delta x_c, \Delta y_c, \Delta \theta_c)$ are explicitly bounded using `np.clip()` to ensure they stay within feasible magnitudes. For instance, the position updates are clipped to ± 2 mm per time step, and angle updates are clipped to a maximum of ± 0.01 radians.

$$\Delta u = \text{clip}(\Delta u, [\min], [\max])$$

- **Trajectory Smoothness:** To prevent large jumps in control due to discretization noise or overshoot, I applied a smoothing term to the derivative of the tracking error using exponential smoothing. This effectively dampens high-frequency control spikes and improves convergence.
- **Trajectory Design:** The desired trajectory for the middle node is carefully chosen to remain within the beam’s feasible workspace. For instance, the reference path is a circular arc with radius $L/2$, ensuring that the midpoint remains reachable throughout the motion.
- **Initialization Matching:** The simulation begins with the beam in a configuration that aligns with the first point on the desired trajectory. This reduces large initial errors that could result in infeasible commands in the first few steps.